



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ИДЗ № 2

**ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки
информации и управления»**

Выполнил: студент гр. ИУ5-23Б

(Подпись) (Титов В. С.)
(Ф.И.О.)

Проверил:

(Подпись) (Гапанюк Ю. Е.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026 г.

Цель: разработать консольное приложение на C# для работы с базой данных SQLite с использованием принципов ООП и формирования отчётов.

Задачи:

- Создать классы-модели для таблиц базы данных
- Реализовать работу с SQLite (создание, чтение, изменение, удаление данных)
- Импортировать данные из CSV-файлов
- Разработать консольное меню для взаимодействия с пользователем
- Реализовать формирование отчётов с использованием Fluent Interface
- Выполнить SQL-запросы с использованием JOIN, GROUP BY, COUNT и AVG

Выполнение работы:

1. Модель данных

В рамках работы были разработаны две сущности:

Ресторан (справочная таблица) - содержит идентификатор и название ресторана.

Блюдо (основная таблица) - содержит идентификатор, внешний ключ на ресторан, название и цену.

Связь между таблицами: "один-ко-многим" (один ресторан может иметь много блюд).

2. Импорт данных из CSV

При первом запуске программа автоматически загружает данные из CSV-файлов:

- restaurants.csv - справочник ресторанов (4 записи)
- menuitems.csv - основная таблица блюд (12 записей)

Формат файлов: разделитель ";", первая строка - заголовки.

3. Работа с базой данных

Класс DatabaseManager обеспечивает:

- создание таблиц
- импорт из CSV
- получение списков записей
- добавление, редактирование и удаление блюд
- выполнение SQL-запросов для отчётов

4. Консольное меню

Реализовано интерактивное меню с циклическим вводом. Доступны следующие функции:

Пункт 1 - Показать все рестораны
Пункт 2 - Показать все блюда
Пункт 3 - Добавить блюдо
Пункт 4 - Редактировать блюдо
Пункт 5 - Удалить блюдо
Пункт 6 - Отчёты
Пункт 7 - Фильтр по ресторану
Пункт 0 - Выход

При добавлении и редактировании блюда:

- выводится список доступных ресторанов
- ввод названия и цены
- проверка корректности ввода

5. Формирование отчётов

Реализован класс ReportBuilder, использующий паттерн Fluent Interface. Методы возвращают this, что позволяет строить цепочки вызовов.

6. Реализованные отчёты

1. Полный список блюд - объединение таблиц (JOIN), сортировка по названию блюд а.
2. Количество блюд по ресторанам - группировка (GROUP BY) с подсчётом (COUNT).
3. Средняя цена по ресторанам - группировка с вычислением среднего (AVG), сортировка по убыванию.

7. Валидация данных

В классе MenuItem в set-аксессоре свойства Price выполняется проверка: если значение отрицательное, генерируется исключение ArgumentException. Это предотвращает запись некорректных данных в базу.

Ответы на контрольные вопросы:

1. Что представляет собой архитектура разработанного приложения?

Приложение реализовано в виде консольной программы с разделением на слои:

- классы-модели (Restaurant, MenuItem) - представляют таблицы БД
- класс DatabaseManager - инкапсулирует работу с SQLite
- класс ReportBuilder - формирование отчётов с Fluent Interface
- основной модуль Program - консольное меню и взаимодействие с пользователем

2. Что такое связь "один-ко-многим"?

Это связь, при которой одной записи из справочной таблицы соответствует несколько записей из основной таблицы. В данной работе: один ресторан может иметь много блюд. Реализуется через внешний ключ (foreign key).

3. Что такое первичный и внешний ключ?

Первичный ключ (Primary Key) - уникальный идентификатор записи в таблице. Внешний ключ (Foreign Key) - поле, ссылающееся на первичный ключ другой таблицы, обеспечивающее целостность связи между таблицами.

4. Зачем используются свойства (properties) вместо полей?

Свойства позволяют:

- контролировать доступ к данным
- добавлять валидацию в set-аксессоре
- инкапсулировать логику внутри класса
- изменять реализацию без изменения интерфейса

5. Как реализована валидация данных?

Валидация реализована в set-аксессоре свойства Price в классе MenuItem. При попытке установить отрицательное значение генерируется ArgumentException с сообщением об ошибке. Исключение перехватывается в консольном меню, и пользователь видит понятное сообщение.

6. Что делает класс DatabaseManager?

Класс инкапсулирует работу с базой данных:

- создание таблиц (CREATE TABLE)
- импорт данных из CSV-файлов

- CRUD-операции (Create, Read, Update, Delete) для блюд
- получение списков ресторанов и блюд
- выполнение произвольных SQL-запросов для отчётов
- фильтрацию блюд по ресторану

7. Зачем используется конструкция using при работе с БД?

Конструкция using обеспечивает автоматическое освобождение ресурсов. При выходе из блока using вызывается метод Dispose(), который закрывает соединение с базой данных. Это предотвращает утечки памяти и гарантирует, что соединение будет закрыто даже при возникновении исключения.

8. Что такое Fluent Interface?

Это шаблон проектирования, при котором методы класса возвращают ссылку на текущий объект (return this), что позволяет вызывать методы цепочкой в одной строке. Это делает код более читаемым и выразительным, напоминая естественный язык.

9. Как работает ReportBuilder?

Класс ReportBuilder последовательно накапливает параметры отчёта (SQL-запрос, заголовки, названия колонок, ширину колонок, опции нумерации и нижнего колонтитула). Терминальный метод Build() выполняет SQL-запрос через DatabaseManager, получает результат, форматирует его с помощью StringBuilder с учётом всех настроек и возвращает готовую строку отчёта. Метод Print() вызывает Build() и выводит результат в консоль.

10. Какие типы SQL-запросов использованы?

- SELECT с JOIN - для объединения таблиц ресторанов и блюд
- GROUP BY - для группировки записей по ресторанам
- COUNT(*) - для подсчёта количества блюд в каждом ресторане
- AVG() - для вычисления средней цены блюд по ресторанам
- ORDER BY - для сортировки результатов
- WHERE - для фильтрации

11. Зачем нужен CSV-импорт?

CSV-импорт используется для начального заполнения базы данных тестовыми данными. При первом запуске программа проверяет, пусты ли таблицы, и если да - загружает данные из CSV-файлов. Это позволяет сразу продемонстрировать работу приложения без необходимости вручную вводить все записи.

12. В чём преимущество разделения на классы?

Разделение на классы:

- улучшает читаемость и структурированность кода
- упрощает поддержку и модификацию
- повышает переиспользуемость компонентов
- реализует принципы ООП (инкапсуляцию, единственную ответственность)
- облегчает тестирование отдельных модулей

13. Что делает консольное меню?

Консольное меню обеспечивает взаимодействие пользователя с программой:

- отображение списка доступных действий
- циклический ввод команды до выбора выхода
- вызов соответствующих функций
- обработку ошибок ввода
- вывод результатов выполнения

14. Какие отчёты реализованы?

Реализованы три обязательных отчёта:

1. Полный список блюд - отображает все блюда с названиями ресторанов (JOIN, сортировка по названию блюда).
2. Количество блюд по ресторанам - показывает, сколько блюд в каждом ресторане (GROUP BY, COUNT).
3. Средняя цена по ресторанам - вычисляет среднюю цену блюд в каждом ресторане, сортировка по убыванию (GROUP BY, AVG, ORDER BY DESC)

Код программы представлен в листинге 1:

```
// Файл: Models/Restaurant.cs
namespace RestaurantMenuApp_Console.Models;

public class Restaurant
{
    public int Id { get; set; }
    public string Name { get; set; }

    public Restaurant(int id, string name)
    {
        Id = id;
        Name = name;
    }
}
```

```

public Restaurant() : this(0, "") { }

public override string ToString() => $"[{Id}] {Name}";
}

// Файл: Models/MenuItem.cs
namespace RestaurantMenuApp_Console.Models;

public class MenuItem
{
    public int Id { get; set; }
    public int RestaurantId { get; set; }
    public string Name { get; set; }

    private decimal _price;

    public decimal Price
    {
        get => _price;
        set
        {
            if (value < 0)
                throw new ArgumentException("Цена не может быть отрицательной");
            _price = value;
        }
    }

    public MenuItem(int id, int restaurantId, string name, decimal price)
    {
        Id = id;
        RestaurantId = restaurantId;
        Name = name;
        Price = price;
    }

    public MenuItem() : this(0, 0, "", 0) { }

    public override string ToString() => $"[{Id}] {Name}, ресторан #{RestaurantId},
цена: {Price:F2} руб.";
}

// Файл: Database/DatabaseManager.cs (основные методы)
public class DatabaseManager
{
    private string _connectionString;

    public DatabaseManager(string dbPath)
    {

```

```

        _connectionString = $"Data Source={dbPath}";
    }

    public void InitializeDatabase(string restaurantsCsv, string menuItemsCsv)
    {
        CreateTables();
        if (GetAllRestaurants().Count == 0 && File.Exists(restaurantsCsv))
            ImportRestaurantsFromCsv(restaurantsCsv);
        if (GetAllMenuItems().Count == 0 && File.Exists(menuItemsCsv))
            ImportMenuItemsFromCsv(menuItemsCsv);
    }

    public List<Restaurant> GetAllRestaurants() { ... }
    public List<MenuItem> GetAllMenuItems() { ... }
    public void AddMenuItem(MenuItem item) { ... }
    public void UpdateMenuItem(MenuItem item) { ... }
    public void DeleteMenuItem(int id) { ... }
    public (string[] columns, List<string[]> rows) ExecuteQuery(string sql) { ... }
}

// Файл: Reports/ReportBuilder.cs
public class ReportBuilder
{
    private DatabaseManager _db;
    private string _sql = "";
    private string _title = "";
    private string[] _headers = Array.Empty<string>();
    private int[] _widths = Array.Empty<int>();
    private bool _numbered = false;
    private string _footer = "";

    public ReportBuilder(DatabaseManager db) { _db = db; }

    public ReportBuilder Query(string sql) { _sql = sql; return this; }
    public ReportBuilder Title(string title) { _title = title; return this; }
    public ReportBuilder Header(params string[] columns) { _headers = columns; return this; }
    public ReportBuilder ColumnWidths(params int[] widths) { _widths = widths; return this; }
    public ReportBuilder Numbered() { _numbered = true; return this; }
    public ReportBuilder Footer(string label) { _footer = label; return this; }

    public string Build() { ... }
    public void Print() { Console.WriteLine(Build()); }
}

// Файл: Program.cs (консольное меню и отчёты)
static void Report1_FullList(DatabaseManager db)

```



```

{
    new ReportBuilder(db)
        .Query(@"SELECT mi.item_name, r.restaurant_name, mi.price
                FROM menu_items mi
                JOIN restaurants r ON mi.restaurant_id = r.restaurant_id
                ORDER BY mi.item_name")
        .Title("Полный список блюд с ресторанами")
        .Header("Блюдо", "Ресторан", "Цена (руб.)")
        .ColumnWidths(30, 30, 15)
        .Numbered()
        .Footer("Всего блюд")
        .Print();
}

```

```

static void Report2_CountByRestaurant(DatabaseManager db)
{
    new ReportBuilder(db)
        .Query(@"SELECT r.restaurant_name, COUNT(*) AS cnt
                FROM menu_items mi
                JOIN restaurants r ON mi.restaurant_id = r.restaurant_id
                GROUP BY r.restaurant_name
                ORDER BY r.restaurant_name")
        .Title("Количество блюд по ресторанам")
        .Header("Ресторан", "Кол-во блюд")
        .ColumnWidths(30, 15)
        .Print();
}

```

```

static void Report3_AvgPriceByRestaurant(DatabaseManager db)
{
    new ReportBuilder(db)
        .Query(@"SELECT r.restaurant_name, ROUND(AVG(mi.price), 2) AS avg_price
                FROM menu_items mi
                JOIN restaurants r ON mi.restaurant_id = r.restaurant_id
                GROUP BY r.restaurant_name
                ORDER BY avg_price DESC")
        .Title("Средняя цена блюд по ресторанам (убывание)")
        .Header("Ресторан", "Средняя цена (руб.)")
        .ColumnWidths(30, 20)
        .Print();
}

```

Вывод:

В ходе выполнения работы разработано консольное приложение на C# для управления ресторанами и блюдами с использованием SQLite. Реализованы классы-модели с валидацией данных, инкапсулирующий работу с БД класс DatabaseManager, а также класс ReportBuilder с паттерном Fluent Interface для построения отчётов. Консольное меню обеспечивает выполнение CRUD-операций и формирование трёх отчётов с использованием SQL-запросов JOIN, GROUP BY, COUNT и AVG. Все поставленные задачи выполнены, программа работает корректно и обрабатывает ошибки ввода пользователя.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Быков, А. Ю. Решение задач на языках программирования Си и Си++ : методические указания / А. Ю. Быков. — Москва : МГТУ им. Н.Э. Баумана, 2017. — 248 с. — ISBN 978-5-7038-4577-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/103505>
2. Каширин, И. Ю. От Си к Си++ : учебное пособие / И. Ю. Каширин, В. С. Новичков. — 2-е изд., стер. — Москва : Горячая линия-Телеком, 2012. — 334 с. — ISBN 978-5-9912-0259-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/5161>
3. Быков А. Ю. Решение задач на языках программирования Си и Си++ : метод. указания к выполнению лаб. работ / Быков А. Ю. ; МГТУ им. Н. Э. Баумана. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2017. - 244 с. : ил. - ISBN 978-5-7038-4577-6.
4. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ Н.Э.Баумана, 2002. <http://progbook.ru/technologiya-programmirovaniya/582-ivanova-tehnologiya-programmirovaniya.html>
5. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2014. - 455 с. : ил. - Библиогр.: с. 450. - ISBN 978-5-7038-3921-8.
6. Подбельский В.В. Язык Си++: Учебное пособие. – М.: Финансы и статистика, 2003. <http://progbook.ru/c/737-podbelskii-programmiovanie-na-yazyke-si.html>.
7. Акулов О.А., Медведев Н.В. Информатика. Базовый курс. – М.: Омега-Л, 2006. <http://razym.ru/naukaobraz/obrazov/151874-akulov-oa-medvedev-nv-informatika-bazovyy-kurs.html>
8. Вычислительные методы и программирование. МГУ им. М.В. Ломоносова. ISSN 1726-3522. Журнал входит в 1-й уровень Белого списка научных журналов Минобрнауки России. <https://num-meth.ru/index.php/journal/index>

Дополнительные материалы

1. Иванова Г.С. Технология программирования: Учебник для вузов. – М.: Изд. МГТУ им. Ахо А.В., Хопкрофт Д.Э., Ульман Д.Д. Структуры данных и алгоритмы. – М., Вильямс, 2003. <http://razym.ru/naukaobraz/obrazov/181547-aho-a-ulman-d-hopkroft-d-struktury-dannyh-i-algoritmy.html>
2. Дейтел Х.М., Дейтел П.Дж. Как программировать на С++. – М.: Бином, 2001.
3. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005. <http://rutracker.org/forum/viewtopic.php?t=533181>
4. Джосьютис Н. С++ Стандартная библиотека для профессионалов. – СПб.: Питер, 2004. http://progbook.ru/c/178-dzhosyutis_s_standartnaya_biblioteka.html
5. Подбельский В.В. Стандартный Си++: Учебное пособие. – М.: Финансы и статистика, 2008.
6. Объектно-ориентированное программирование в С++: пер. с англ. / Лафоре Р. - 4-е изд. - СПб.: Питер, 2004. - 923 с. - (Классика computer science). - ISBN 5-94723-302-9.
7. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005.
8. Г. Шилдт. С++. Базовый курс, 3-е издание: Пер с англ. – М.: Издательский дом «Вильямс», 2011. – 624 с.
9. Павловская Т. А. С/С++. Программирование на языке высокого уровня: учебник для вузов / Павловская Т. А. - СПб.: Питер, 2003. - 460 с. - (Учебник для вузов). - ISBN 5-94723-568-4.
10. Бесплатные образовательные программы партнера (VK): <https://education.vk.company/students>